

Digital Twin Data Transmission: HTTP REST and MQTT Demonstration

1. Introduction & Justification

To build an effective digital twin of a vehicle (e.g., a Mercedes-Benz W203 C200 Kompressor), we must synchronize the physical asset's state with its digital counterpart using continuous data streams. This notebook demonstrates the transmission and reception of two critical types of digital data:

1. **Telemetry Data (Numbers):** Engine temperature (°C) and Bosch fuel pump pressure (kPa).
2. **Spatial Data (Coordinates):** GPS Latitude and Longitude.

Data Format Justification

All data is formatted as **JSON (JavaScript Object Notation)**.

- **Justification:** JSON is a lightweight, human-readable data-interchange format. It is universally supported by both HTTP REST APIs and MQTT message brokers, making it the industry standard for IoT and digital twin ecosystems. It allows us to easily serialize complex nested data (like coordinates and timestamps) into a string payload and deserialize it back into programmable objects.

Protocol Route/Interface & Trigger Justification

- **HTTP REST (Asynchronous Updates):** Best suited for state updates that require immediate confirmation of receipt (Request-Response model).
 - *Route/Interface:* POST `/api/telemetry/{sensor_id}`
 - *Trigger:* Triggered asynchronously on a set interval or when a significant state change occurs (e.g., pressure drop).
- **MQTT (Streaming):** Best suited for high-frequency, continuous data streams like GPS tracking, where low overhead and publish/subscribe architecture (PubSub) are necessary.
 - *Route/Interface:* Topic `vehicle/w203/gps/{tracker_id}`
 - *Trigger:* Triggered continuously by a background time-series loop to stream real-time location.

```
# Setup and Imports
import asyncio
import json
import random
import time
from aiohttp import web, ClientSession
import paho.mqtt.client as mqtt

# Global dictionaries to store received data for verification
received_http_data = {}
received_mqtt_data = {}

# Configuration
HTTP_HOST = '127.0.0.1'
HTTP_PORT = 8082 # Changed from 8081 to avoid 'address already in use' error
# Using a public MQTT broker for demonstration purposes
MQTT_BROKER = "broker.hivemq.com"
MQTT_PORT = 1883
MQTT_TOPIC_BASE = "vehicle/w203/gps/"
```

```
# Data Generation Functions
def generate_telemetry_data(sensor_type):
    """Generates random numerical telemetry data for HTTP REST."""
    if sensor_type == "bosch_fuel_pump":
        return {
            "timestamp": time.time(),
            "sensor": sensor_type,
            "pressure_kpa": round(random.uniform(350.0, 400.0), 2),
            "status": "optimal"
        }
    elif sensor_type == "engine_temp":
        return {
            "timestamp": time.time(),
            "sensor": sensor_type,
            "temperature_c": round(random.uniform(85.0, 95.0), 2),
            "status": "normal"
        }

def generate_gps_data(tracker_id):
    """Generates random coordinate data for MQTT streaming."""
    # Base coordinates somewhere in Malaysia
    base_lat, base_lon = 4.35, 100.98
    return {
        "timestamp": time.time(),
        "tracker": tracker_id,
        "latitude": round(base_lat + random.uniform(-0.01, 0.01), 6),
        "longitude": round(base_lon + random.uniform(-0.01, 0.01), 6)
    }
```

2. HTTP REST Implementation (Asynchronous Updates)

Here we define a local asynchronous web server to act as the Digital Twin backend receiving data. We also define the HTTP client that will trigger the `POST` requests to update the twin with data from two different sensors.

```
# HTTP REST Server (Digital Twin Backend)
async def handle_telemetry_post(request):
    """Endpoint to receive telemetry data."""
    sensor_id = request.match_info.get('sensor_id')
    try:
        data = await request.json()
        received_http_data[sensor_id] = data
        return web.Response(text=f"Data received for {sensor_id}", status=200)
    except Exception as e:
        return web.Response(text=str(e), status=400)

async def start_http_server():
    app = web.Application()
    app.add_routes([web.post('/api/telemetry/{sensor_id}', handle_telemetry_post)])
    runner = web.AppRunner(app)
    await runner.setup()
    site = web.TCPSite(runner, HTTP_HOST, HTTP_PORT)
    await site.start()
    return runner

# HTTP REST Client (Physical Asset Transmitting)
async def http_transmit(sensor_id, session, payload):
    url = f"http://{HTTP_HOST}:{HTTP_PORT}/api/telemetry/{sensor_id}"
    print(f"[HTTP TRG] Triggering REST POST to {url}")
    async with session.post(url, json=payload) as response:
        status = await response.text()
        print(f"[HTTP ACK] Status: {status}")
```

3. MQTT Implementation (Data Streaming)

Here we configure the MQTT client. The Digital Twin subscribes to the relevant GPS topics to listen for streams, while the physical asset instances will publish their generated coordinate data to those same topics.

```
# MQTT Callback Setup (Digital Twin Backend)
def on_mqtt_connect(client, userdata, flags, rc):
    print(f"[MQTT] Connected to broker with result code {rc}")
    client.subscribe(f"{MQTT_TOPIC_BASE}#")
```

```

def on_mqtt_message(client, userdata, msg):
    """Callback when a stream message is received."""
    topic = msg.topic
    payload = json.loads(msg.payload.decode('utf-8'))
    received_mqtt_data[topic] = payload
    print(f"[MQTT RCV] Stream received on {topic}")

# Initialize MQTT Client
mqtt_client = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2, client_id=f"dt_list")
mqtt_client.on_connect = on_mqtt_connect
mqtt_client.on_message = on_mqtt_message

def start_mqtt_listener():
    mqtt_client.connect(MQTT_BROKER, MQTT_PORT, 60)
    mqtt_client.loop_start()

# MQTT Publisher (Physical Asset Transmitting)
async def mqtt_stream(tracker_id, payload):
    topic = f"{MQTT_TOPIC_BASE}{tracker_id}"
    print(f"[MQTT TRG] Streaming data to topic {topic}")
    mqtt_client.publish(topic, json.dumps(payload))
    await asyncio.sleep(0.1) # Simulate network delay

```

✓ 4. Parallel Execution & Correctness Verification

This final block fulfills the requirement to transmit and receive all data in parallel asynchronously, and subsequently verifies that the data received by the Digital Twin perfectly matches the randomly generated physical asset data.

```

async def run_parallel_simulation():
    print("--- STARTING PARALLEL TRANSMISSION ---\n")

    # 1. Start Servers / Listeners
    http_runner = await start_http_server()
    start_mqtt_listener()
    await asyncio.sleep(3) # Allow connections to establish

    # 2. Generate Data from 2 sources for each protocol
    http_payload_1 = generate_telemetry_data("bosch_fuel_pump")
    http_payload_2 = generate_telemetry_data("engine_torque")

    mqtt_payload_1 = generate_gps_data("tracker_front")
    mqtt_payload_2 = generate_gps_data("tracker_rear")

    # 3. Transmit all data in PARALLEL
    async with ClientSession() as session:
        tasks = [
            http_transmit("bosch_fuel_pump", session,

```

```

        http_transmit("engine_temp", session, http_payload_1)
        mqtt_stream("tracker_front", mqtt_payload_1)
        mqtt_stream("tracker_rear", mqtt_payload_2)
    ]
    await asyncio.gather(*tasks)

await asyncio.sleep(3) # Wait briefly for background tasks to complete

print("\n--- COMPLETION & VERIFICATION ---")

# 4. Verify HTTP REST correctness
print("\n[Verification] Checking HTTP REST Data...")
assert received_http_data["bosch_fuel_pump"] == http_payload_1
assert received_http_data["engine_temp"] == http_payload_2
print(f"SENT Fuel Pump: {http_payload_1}")
print(f"RCVD Fuel Pump: {received_http_data['bosch_fuel_pump']}")
print("SUCCESS: HTTP REST data successfully verified")

# 5. Verify MQTT correctness
print("\n[Verification] Checking MQTT Stream Data...")
assert received_mqtt_data[f"{MQTT_TOPIC_BASE}tracker_front"] == mqtt_payload_1
assert received_mqtt_data[f"{MQTT_TOPIC_BASE}tracker_rear"] == mqtt_payload_2
print(f"SENT Tracker Front: {mqtt_payload_1}")
print(f"RCVD Tracker Front: {received_mqtt_data[f'{MQTT_TOPIC_BASE}tracker_front']}")
print("SUCCESS: MQTT stream data successfully verified")

# Cleanup
await http_runner.cleanup()
mqtt_client.loop_stop()
mqtt_client.disconnect()

# Execute the main parallel simulation block
await run_parallel_simulation() # Re-executing to pick up from where we left off

```

--- STARTING PARALLEL TRANSMISSION ---

Exception in thread paho-mqtt-client-dt_listener_3644:

Traceback (most recent call last):

File "/usr/lib/python3.12/threading.py", line 1075, in _bootstrap_inner
self.run()

File "/usr/lib/python3.12/threading.py", line 1012, in run
self._target(*self._args, **self._kwargs)

File "/usr/local/lib/python3.12/dist-packages/paho/mqtt/client.py", line 4523, in loop_forever
self.loop_forever(retry_first_connection=True)

File "/usr/local/lib/python3.12/dist-packages/paho/mqtt/client.py", line 2297, in _loop(timeout)
rc = self._loop(timeout)
^^

File "/usr/local/lib/python3.12/dist-packages/paho/mqtt/client.py", line 1686, in _loop_read()
rc = self._loop_read()
^^

File "/usr/local/lib/python3.12/dist-packages/paho/mqtt/client.py", line 2100, in _packet_read()
rc = self._packet_read()
^^

